

FastAPI MVP – Take-Home Exam

This competency-based exam assesses your ability to design, implement, and test a focused Minimum Viable Product (MVP) for a simple RESTful API using the FastAPI framework. You will create an API to manage simple items in-memory.

You are tasked with building the backend API for managing a collection of simple items. Users should be able to create new items and retrieve specific items by their ID. For this MVP, data persistence will be handled in memory (no database is required).

Requirements

1. Project Setup: Set up a new Python project with a virtual environment. Install necessary libraries: fastapi, uvicorn, pydantic, pytest, httpx. Create a main application file (e.g., main.py).

2. Data Modeling (Pydantic): Define a pydantic model BaseModel with the following fields:

- name (string, required)
- description (string, optional, default=None)
- value (float, required)

Define another pydantic Item that extends the BaseModel and adds an id (integer, required).

3. In-Memory Storage: Use a Python dictionary to store items in memory. The key should be the item ID, and the value should be the item data (as a dictionary or pydantic model instance). Maintain a simple counter for generating unique item IDs.

4. API Endpoints (FastAPI): Implement *only* the following two endpoints:

- **POST /items/:** Creates a new item.
 - Accepts a BaseModel model in the request body, then assigns a unique ID to the new item.
 - Stores the Item in the in-memory dictionary and returns the newly created Item (including its ID) with a 201 Created status code.
- **GET /items/{item_id}:** Retrieves a specific item by its ID.
 - Accepts item_id as a path parameter.
 - Returns the corresponding Item model.
 - If the item is not found, return a 404 Not Found HTTP exception.

5. Error Handling: Implement proper HTTP exceptions via HTTPException for the "Item not found" (404) case in the GET /items/{item_id} endpoint.

6. Testing: Create a separate test file (e.g., `test_main.py` or `test_main.sh`). Use FastAPI's `TestClient` or any other approach to write integration tests for the implemented API endpoints. Your tests should cover:

- Successful creation and retrieval.
- Retrieving a non-existent item (expecting 404).
- Validation errors during creation (e.g., missing required fields - FastAPI handles this, but test it).

7. Documentation: Ensure your code is well-commented. Include your test results in the documentation (see below).

Submission

Submit your code as a compressed archive (.zip or .tar.gz). Include the following items:

- All Python files (`main.py`, `test_main.py`, etc.).
- A `requirements.txt` file listing the dependencies (e.g., `fastapi`, `uvicorn`, `pytest`, `httpx`).
- A brief `README.md` file explaining how to set up the virtual environment, install dependencies, and run the application and tests.

Grading Criteria

- **Functionality (45%):** The two required endpoints work correctly as specified. Data is managed correctly in memory. Error handling is implemented for the GET endpoint.
- **Code Quality (20%):** Code is clean, well-organized, and comments are included.
- **Testing (20%):** Comprehensive test coverage for the two implemented endpoints and their specified scenarios. Tests are well-written and pass.
- **Documentation & Setup (15%):** `requirements.txt` is accurate. `README.md` provides clear instructions. Automatic API documentation is functional.